

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y Diseño de Software NRC:30723
	Apellidos y nombres del estudiante: • Carlos Daniel Caiza Quinga • Kerlly Alexandra Chiriboga Cabrera • Alexander Francisco Villacres Guerrero

ANTECEDENTES

El sistema Bocatto es una solución tecnológica diseñada para modernizar y optimizar la gestión de pedidos dentro del restaurante Bocatto Valley. Actualmente, el establecimiento enfrenta una problemática de gestión ineficiente, caracterizada por errores en la toma de órdenes manuales, demoras en la comunicación entre el salón y la cocina, y una falta de visibilidad sobre el estado real de los pedidos en tiempo real. Estos factores no solo generan retrasos operativos, sino que afectan directamente la experiencia del cliente.

Desde la perspectiva de la Ingeniería de Software, el proyecto se desarrolla bajo la metodología Agile Unified Process (AUP), la cual promueve la construcción incremental del sistema mediante iteraciones cortas y entregas continuas de valor. Para responder a los desafíos de escalabilidad, mantenibilidad y organización del código, se propone la implementación de una arquitectura basada en el patrón Modelo-Vista-Controlador (MVC). Esta arquitectura permite separar claramente las responsabilidades de presentación, lógica de negocio y acceso a datos, facilitando el desarrollo colaborativo, la reutilización de componentes y la evolución progresiva del sistema conforme crecen los requerimientos del restaurante.

Dentro de esta estructura arquitectónica, se plantea además la incorporación de patrones de diseño creacionales que complementen la organización del software. El patrón Builder permitirá gestionar la construcción de pedidos complejos mediante un proceso gradual y controlado, adaptándose a distintos canales de venta como atención en mesa, pedidos telefónicos o pedidos mediante código QR. Por su parte, el patrón Prototype facilitará la creación eficiente de promociones, combos y configuraciones de menú mediante la clonación de instancias previamente definidas, reduciendo la carga de procesamiento y simplificando la gestión de elementos recurrentes.

La combinación de la arquitectura MVC con los patrones Builder y Prototype busca proporcionar una solución robusta y flexible que contribuya a superar los problemas de gestión actuales del restaurante. Asimismo, esta propuesta favorece la calidad del software al promover el desacoplamiento de componentes, la reutilización de código, la facilidad de mantenimiento y la adaptación a futuros cambios en los procesos operativos del negocio.

OBJETIVOS

- OBJETIVO GENERAL

- Justificar técnica y operativamente la implementación de la arquitectura Modelo-Vista-Controlador (MVC) y de los patrones de diseño creacionales Builder y Prototype en el desarrollo del sistema de gestión de pedidos Bocatto Valley, con el fin de mejorar la organización del software, optimizar la creación de objetos complejos y garantizar la escalabilidad, mantenibilidad y calidad del sistema.

- OBJETIVOS ESPECÍFICOS

- Describir cómo la arquitectura MVC permite separar las responsabilidades de presentación, lógica de negocio y acceso a datos, favoreciendo la mantenibilidad, escalabilidad y reutilización de componentes dentro del sistema Bocatto.
- Detallar cómo el patrón Builder facilita la construcción gradual de pedidos provenientes de diferentes canales de atención (mesa, teléfono y código QR), garantizando la correcta inicialización de los datos y el cumplimiento de las reglas de negocio.
- Aplicar el patrón Prototype para permitir la clonación eficiente de promociones, combos y configuraciones de menú previamente definidas, reduciendo el costo de creación de objetos similares y mejorando el rendimiento del sistema.
- Proponer una solución arquitectónica y de diseño que contribuya a reducir la complejidad del software mediante el desacoplamiento de componentes y la utilización de patrones probados en la industria.
- Evaluar los beneficios que aporta la combinación de MVC, Builder y Prototype en términos de mantenibilidad, escalabilidad, rendimiento y adaptabilidad frente a futuros cambios en los procesos operativos del restaurante.
- Servir como guía para la toma de decisiones técnicas del proyecto, permitiendo validar la viabilidad de las soluciones propuestas dentro del enfoque iterativo e incremental de la metodología Agile Unified Process (AUP).

JUSTIFICACIÓN DE LA ARQUITECTURA MVC (MODELO – VISTA – CONTROLADOR)

La arquitectura Modelo-Vista-Controlador (MVC) es un patrón arquitectónico ampliamente utilizado en el desarrollo de aplicaciones empresariales debido a su capacidad para separar las responsabilidades del sistema en componentes independientes. Su principal objetivo es desacoplar

la interfaz de usuario de la lógica de negocio y de la gestión de datos, facilitando el mantenimiento, la escalabilidad y la evolución del software.

Aplicación de MVC en el Sistema Bocatto

El sistema Bocatto debe gestionar múltiples procesos relacionados con la administración de pedidos, menús digitales, promociones y seguimiento de órdenes en tiempo real. Además, interactúan diferentes actores, como clientes, meseros, operadores telefónicos, personal de cocina y administradores. Debido a esta complejidad, resulta necesario adoptar una arquitectura que permita organizar adecuadamente las responsabilidades de cada componente.

En este contexto, la arquitectura MVC distribuye las funciones del sistema de la siguiente manera:

- **Modelo (Model):** contiene las entidades del dominio, como Pedido, Menu, Promocion, Cliente y Usuario, además de las reglas de negocio y los mecanismos de acceso a datos.
- **Vista (View):** corresponde a las interfaces utilizadas por los diferentes actores para registrar pedidos, consultar el menú digital, administrar promociones y visualizar el estado de las órdenes.
- **Controlador (Controller):** actúa como intermediario entre la vista y el modelo, procesando las solicitudes de los usuarios y coordinando las operaciones necesarias para cumplir los requerimientos del sistema.

Esta separación permite que las modificaciones realizadas en la interfaz de usuario no afecten la lógica de negocio ni la persistencia de los datos, reduciendo el impacto de los cambios durante las iteraciones de desarrollo.

Beneficios de MVC para el Proyecto

La adopción de la arquitectura MVC aporta beneficios significativos para el desarrollo del sistema Bocatto:

1. **Separación de responsabilidades:** cada componente tiene funciones claramente definidas, reduciendo el acoplamiento entre módulos.
2. **Mayor mantenibilidad:** los cambios en la presentación, la lógica de negocio o la base de datos pueden realizarse de forma independiente.
3. **Escalabilidad del sistema:** facilita la incorporación de nuevas funcionalidades, como módulos de reportes, métricas o nuevos canales de atención.
4. **Facilidad de pruebas:** la independencia entre capas permite realizar pruebas unitarias y de integración con mayor facilidad.
5. **Reutilización de componentes:** la lógica del modelo puede ser utilizada por diferentes interfaces sin necesidad de duplicar código.
6. **Compatibilidad con AUP:** favorece el desarrollo incremental e iterativo propuesto por la metodología Agile Unified Process, permitiendo implementar mejoras progresivas sin afectar el funcionamiento global del sistema.

JUSTIFICACIÓN DEL PATRÓN BUILDER

El patrón Builder se fundamenta en la separación de la lógica de construcción de un objeto complejo de su representación final, permitiendo que el mismo proceso de construcción pueda dar lugar a diferentes configuraciones. En el contexto de Bocatto, este patrón es esencial para gestionar la creación de la entidad central del negocio: el Pedido.

Aplicación en el Proceso de Creación de Pedidos

La documentación de los casos de uso extendidos revela que la creación de un pedido no es un evento instantáneo, sino un proceso incremental y paso a paso:

- **Construcción Incremental:** En los casos de uso CU 1.1 (Registrar pedido en mesa), CU 2.1 (Registrar pedido telefónico) y CU 3.2 (Registrar pedido por QR), el actor debe seleccionar productos y definir cantidades de manera repetitiva hasta completar la orden. El patrón Builder permite que el objeto "Pedido" se vaya conformando dinámicamente a medida que el mesero, el operador o el cliente interactúa con la interfaz.
- **Gestión de Campos Opcionales:** Un pedido en Bocatto incluye múltiples campos que pueden o no estar presentes, como las "Observaciones del pedido" (ej. "sin cebolla"). Además, dependiendo del canal, puede requerir datos de contacto y dirección de entrega (pedido telefónico) o un identificador de mesa (pedido por QR o mesa). El Builder evita la necesidad de "constructores monstruo" con excesivos parámetros, permitiendo inicializar solo los atributos necesarios para cada caso específico.

La implementación de este patrón aporta beneficios críticos para la integridad del software:

1. **Validación Centralizada:** El sistema exige validar que el pedido tenga al menos un producto antes de asignarle un identificador único y el estado "pendiente". Mediante el método build(), el sistema puede centralizar estas validaciones de integridad, asegurando que no se registren órdenes vacías o incompletas en la base de datos.
2. **Independencia de Representación:** Aunque el proceso de construcción de agregar productos es similar, el resultado final puede variar entre un pedido presencial o uno para entrega a domicilio. El Builder permite que el código de construcción sea independiente de estas variaciones, facilitando la escalabilidad del sistema.
3. **Legibilidad y Mantenimiento:** Al separar la lógica de construcción de la lógica de negocio, se cumple con el Principio de Responsabilidad Única, haciendo que el código sea más fácil de leer, probar y mantener frente a futuros cambios en el menú o en las reglas de negocio.

JUSTIFICACIÓN DEL PATRÓN PROTOTYPE (PROTOTIPO)

El patrón Prototype es un patrón de diseño creacional que permite la creación de nuevos objetos a partir de la clonación de instancias existentes, denominadas prototipos. En el sistema Bocatto, este patrón se vuelve fundamental para manejar la variabilidad y repetición de elementos en el menú digital y las promociones.

Aplicación en la Gestión de Menú y Promociones

El sistema cuenta con un módulo de Administración de configuración (CU 0-7) donde el administrador gestiona el menú digital, precios y promociones vigentes. La aplicación del patrón Prototype se centra en:

- **Clonación de Combos y Promociones:** Según el requerimiento RF-08, el administrador debe gestionar promociones y combos. Estos elementos suelen tener configuraciones base (productos incluidos, descuentos y condiciones) que se repiten con frecuencia. En lugar de instanciar cada pedido de promoción desde cero, el sistema puede clonar un "prototipo" de combo ya configurado, ahorrando tiempo de procesamiento.
- **Menú Digital Dinámico:** El cliente accede al menú mediante un código QR (CU 3.1), visualizando productos y promociones actualizadas. El uso de prototipos permite que el sistema añada o elimine productos dinámicamente en tiempo de ejecución, facilitando una estructura flexible que se adapta a los cambios diarios en la disponibilidad del restaurante.

La adopción de este patrón responde a necesidades de eficiencia detectadas en el análisis del proyecto:

1. **Optimización del Rendimiento:** La creación de objetos mediante la copia de instancias es significativamente más rápida que la instanciación tradicional (usando new) cuando el costo de inicialización es elevado debido a múltiples campos o relaciones complejas.
2. **Reducción de la Jerarquía de Clases:** Al delegar la autorreplicación al propio objeto, se evita la necesidad de crear múltiples subclases de fábrica para cada tipo de producto o combo, simplificando la arquitectura del controlador.
3. **Gestión de Estados Preconfigurados:** El patrón permite evitar la "inicialización laboriosa" de campos al copiar prototipos que ya tienen valores definidos por el administrador. Esto es especialmente útil en las "Observaciones del pedido" predefinidas o configuraciones estándar de platos que los clientes solicitan repetidamente.
4. **Flexibilidad en Tiempo de Ejecución:** A diferencia de otros patrones, Prototype permite que el sistema registre nuevos tipos de productos o promociones dinámicamente, lo cual es ideal para un restaurante que ajusta su oferta según la temporada o la demanda.

BENEFICIOS ESPERADOS PARA EL PROYECTO

La implementación de la arquitectura MVC junto con los patrones de diseño Builder y Prototype permitirá abordar tanto los problemas operativos del restaurante Bocatto Valley como los desafíos técnicos asociados al desarrollo y mantenimiento del software. La combinación de estas soluciones contribuirá a mejorar la calidad del sistema, optimizar los procesos de atención y facilitar la evolución del producto en futuras iteraciones.

Organización y Mantenibilidad del Software

La adopción de la arquitectura MVC permitirá separar claramente las responsabilidades entre la interfaz de usuario, la lógica de negocio y la gestión de datos.

- **Separación de Responsabilidades**
 - La capa Vista gestionará exclusivamente la interacción con los usuarios del sistema.
 - La capa Controlador coordinará las solicitudes y el flujo de información.
 - La capa Modelo administrará las entidades del negocio, las reglas de validación y el acceso a los datos.
- **Facilidad de Mantenimiento**
 - Las modificaciones realizadas en una capa tendrán un impacto mínimo sobre las demás.
 - Los cambios futuros en las interfaces o en las reglas de negocio podrán implementarse sin afectar la arquitectura general.
 - Se reduce la complejidad del código y la deuda técnica a largo plazo.

Garantía de Integridad y Validación de Datos

Mediante la aplicación del patrón Builder, el sistema garantizará que los pedidos sean contruidos de manera controlada y cumplan con las reglas de negocio antes de ser procesados.

- **Validaciones Centralizadas:** El método build() permite centralizar validaciones críticas, como asegurar que un pedido contenga al menos un producto antes de ser procesado. Esto evita estados inconsistentes en la base de datos y reduce los errores manuales que actualmente afectan al restaurante.
- **Construcción Segura:** Se garantiza que el cliente o el personal no accedan a un objeto "Pedido" incompleto, ya que el patrón aísla el código de construcción de la lógica de negocio.

Optimización del Rendimiento Operativo

La utilización del patrón Prototype permitirá gestionar eficientemente promociones, combos y configuraciones recurrentes del menú.

- **Clonación Eficiente:** La creación de pedidos basados en combos o promociones preconfiguradas mediante la clonación de instancias existentes es significativamente más rápida que la instanciación tradicional desde cero. Esto mejora los tiempos de respuesta del aplicativo web, cumpliendo con los requisitos de rendimiento esperados.

- **Reducción de Carga Computacional:** Al evitar la inicialización laboriosa de campos repetitivos (como precios base o descripciones de productos), se optimiza el uso de recursos del servidor.

Escalabilidad y Mantenibilidad del Código

La combinación de MVC, Builder y Prototype proporciona una base sólida para el crecimiento futuro del sistema.

- Incorporación de Nuevas Funcionalidades
 - Nuevos módulos podrán integrarse sin afectar significativamente los componentes existentes.
 - Será posible añadir nuevos canales de atención, reportes o servicios complementarios manteniendo la organización del código.
- Compatibilidad con la Metodología AUP
 - La arquitectura facilita el desarrollo incremental e iterativo.
 - Permite realizar mejoras continuas durante los diferentes ciclos de construcción del proyecto.

Mejora de la Experiencia Operativa del Restaurante

La solución propuesta impactará directamente en la eficiencia de los procesos internos y en la calidad del servicio ofrecido a los clientes.

- Gestión Eficiente de Pedidos
 - Reducción de errores en la toma y registro de órdenes.
 - Mejor seguimiento del estado de los pedidos entre salón, cocina y caja.
 - Mayor rapidez en la atención al cliente.
- Flexibilidad en la Gestión del Menú
 - Actualización dinámica de productos, promociones y combos.
 - Adaptación rápida a cambios de disponibilidad o estrategias comerciales del restaurante.

Conclusiones

Para finalizar la propuesta técnica del sistema Bocatto, se presentan las siguientes conclusiones basadas en el análisis de los requerimientos funcionales y en la evaluación de la arquitectura y patrones de diseño seleccionados:

- **Arquitectura organizada y mantenible:** La implementación de la arquitectura Modelo-Vista-Controlador (MVC) proporciona una estructura clara para separar las responsabilidades de presentación, lógica de negocio y acceso a datos. Esta organización facilita el mantenimiento del software, reduce el acoplamiento entre componentes y permite que el sistema evolucione de manera ordenada conforme aumenten las necesidades operativas del restaurante.
- **Eficacia en la construcción de pedidos complejos:** El patrón Builder resulta adecuado para gestionar el proceso de creación de pedidos en los diferentes canales de atención (mesa, teléfono y código QR), ya que permite construir objetos

de forma gradual, validada y segura. Esto contribuye a garantizar la integridad de la información antes de que los pedidos sean procesados por las demás áreas del negocio.

- **Eficiencia operativa mediante la clonación de objetos:** El patrón Prototype permite reutilizar configuraciones previamente definidas de promociones, combos y elementos del menú, reduciendo el esfuerzo de inicialización y optimizando el rendimiento del sistema. Esta capacidad aporta flexibilidad para gestionar cambios frecuentes en la oferta comercial del restaurante.
- **Escalabilidad y adaptabilidad del sistema:** La combinación de MVC, Builder y Prototype establece una base tecnológica sólida que facilita la incorporación de nuevas funcionalidades, módulos y procesos sin afectar significativamente la estructura existente. Esto resulta especialmente importante dentro del enfoque iterativo e incremental promovido por la metodología Agile Unified Process (AUP).
- **Mejora de la calidad del servicio:** La aplicación conjunta de la arquitectura y los patrones seleccionados contribuye a reducir errores en el registro de pedidos, mejorar la comunicación entre las áreas involucradas y optimizar los tiempos de respuesta del sistema. Como resultado, se fortalece la experiencia del cliente y se incrementa la eficiencia operativa del restaurante Bocatto Valley.
- **Reducción de la complejidad y la deuda técnica:** El uso de soluciones arquitectónicas y patrones ampliamente reconocidos en la industria permite desarrollar un software más legible, reutilizable y fácil de probar. Esto disminuye los costos de mantenimiento futuro y favorece la sostenibilidad del proyecto a largo plazo.

Sangolquí, a 03 de junio de 2026